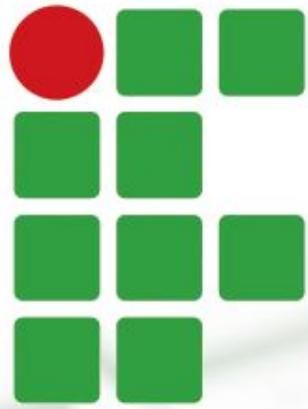




INSTITUTO FEDERAL

Norte de Minas Gerais

Campus Januária

Estruturas de Dados 2

- Generic Lists -



INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Situação Problema





INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Situação Problema

Imagine o desenvolvimento de uma engine para jogo MMO-RPG.





INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Situação Problema

Imagine o desenvolvimento de uma engine para jogo MMO-RPG.

Teremos que pensar em estruturas de dados para representar: **Jogadores Online**, **Inimigos**, **Itens coletados**, etc...





INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Situação Problema

Imagine o desenvolvimento de uma engine para jogo MMO-RPG.

Teremos que pensar em estruturas de dados para representar: **Jogadores Online**, **Inimigos**, **Itens coletados**, etc...

Estruturas Dinâmicas seriam essenciais para melhor desempenho.





Problema Atual

- **Problema Atual**: Existe um **forte acoplamento** entre a ***Entidade*** (objetos que queremos representar) e a ***Estrutura de Dados*** adotada para armazená-la (*p.ex.* Lista Encadeada).



Problema Atual

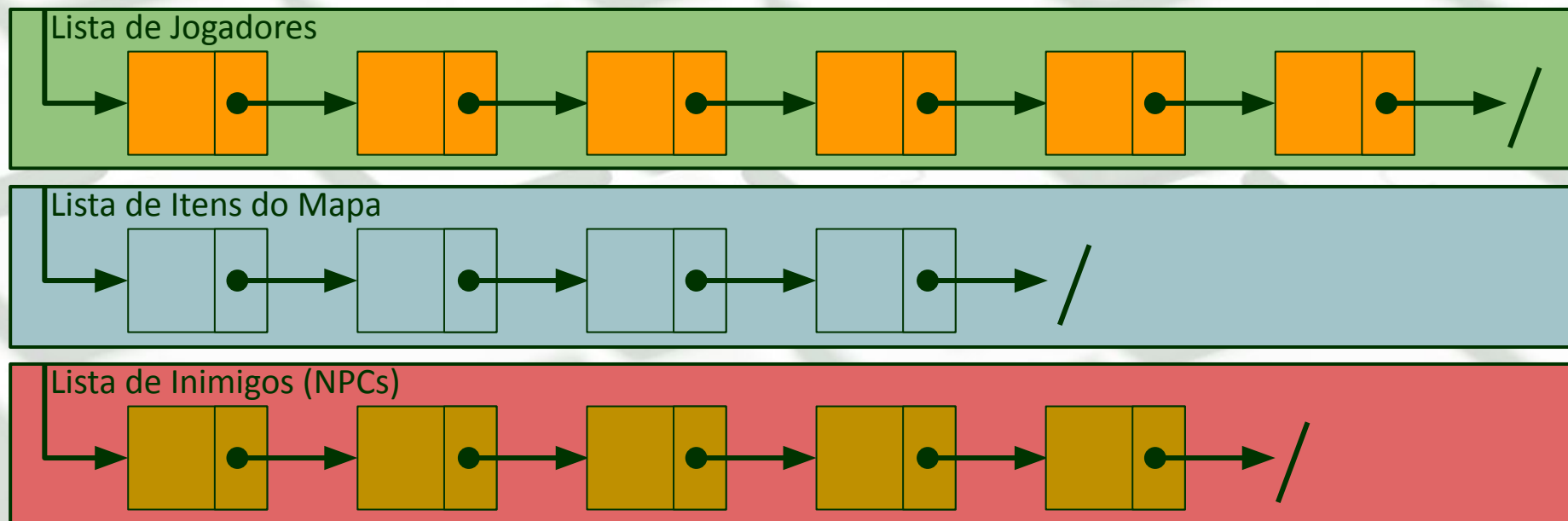
- **Problema Atual**: Existe um **forte acoplamento** entre a **Entidade** (objetos que queremos representar) e a **Estrutura de Dados** adotada para armazená-la (*p.ex.* Lista Encadeada).
- Para tornar mais claro, imagine a seguinte solução...

Faça um programa que armazene, em listas dinâmicas, as seguintes estruturas: Jogadores, Itens do Mapa, Inimigos NPCs.



Problema Atual

- Perceba na solução que, **ao invés de implementar uma única Estrutura de Dados do tipo Lista Encadeada**, implementamos 03 tipos distintos de Lista Encadeada...





Problema Atual

- Perceba que as operações (algoritmos) para manutenção das listas são as mesmas, independentemente da **entidade**, mas implementamos cada operação em cada entidade separadamente...

createPlayer()
<Insere na Lista>

printPlayer()
<Consulta Lista>

editPlayer()
<Edita Lista>

deletePlayer()
<Exclui da Lista>

createItem()
<Insere na Lista>

printItem()
<Consulta Lista>

editItem()
<Edita Lista>

deleteItem()
<Exclui da Lista>

createNPC()
<Insere na Lista>

printNPC()
<Consulta Lista>

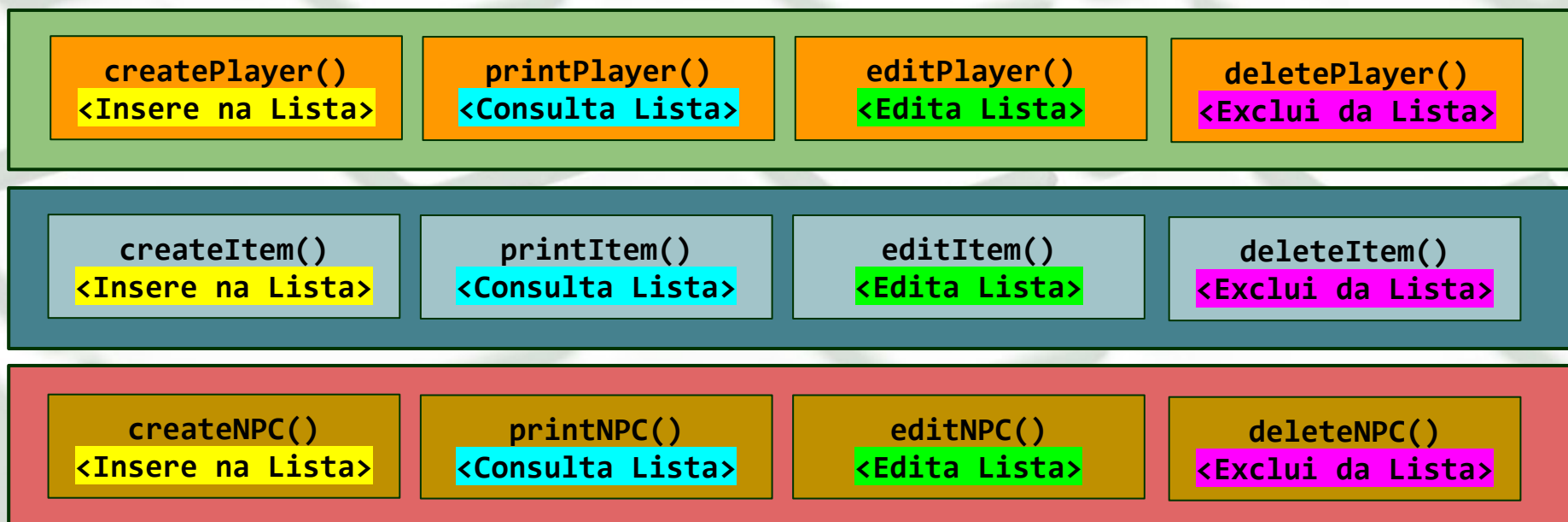
editNPC()
<Edita Lista>

deleteNPC()
<Exclui da Lista>



Problema Atual

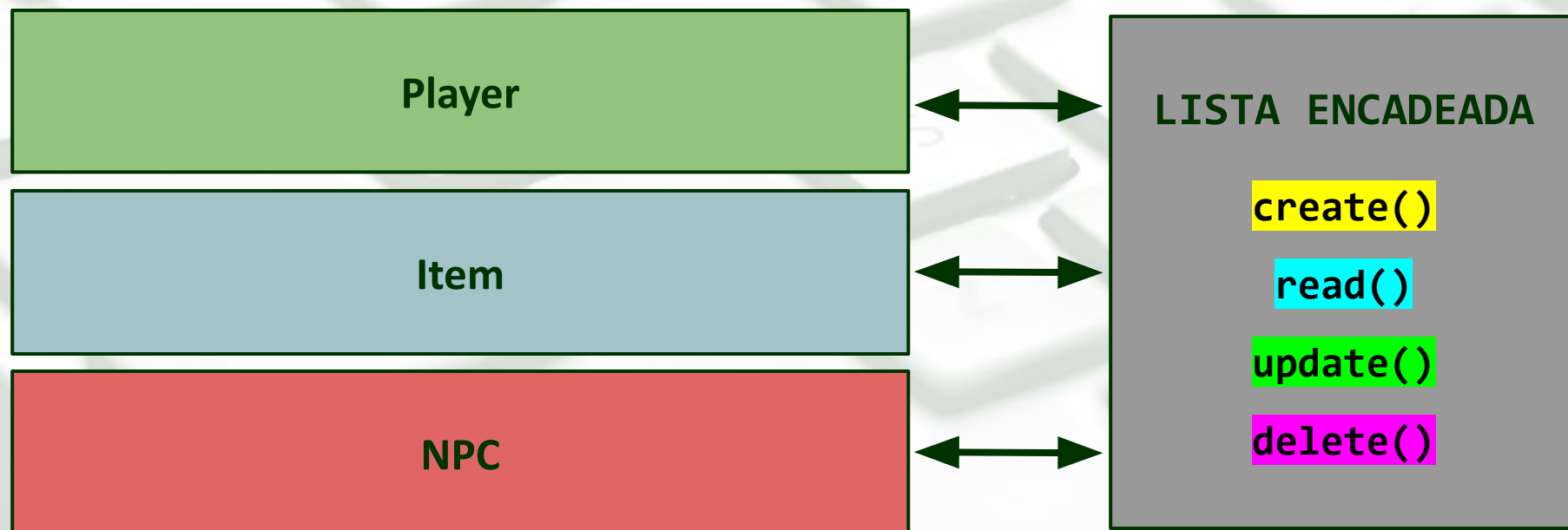
- Perceba que as operações (algoritmos) para manutenção das listas são as mesmas, independentemente da **entidade**, mas implementamos cada operação em cada entidade separadamente... Isso é **ALTO ACOPLAMENTO**.





Problema Atual

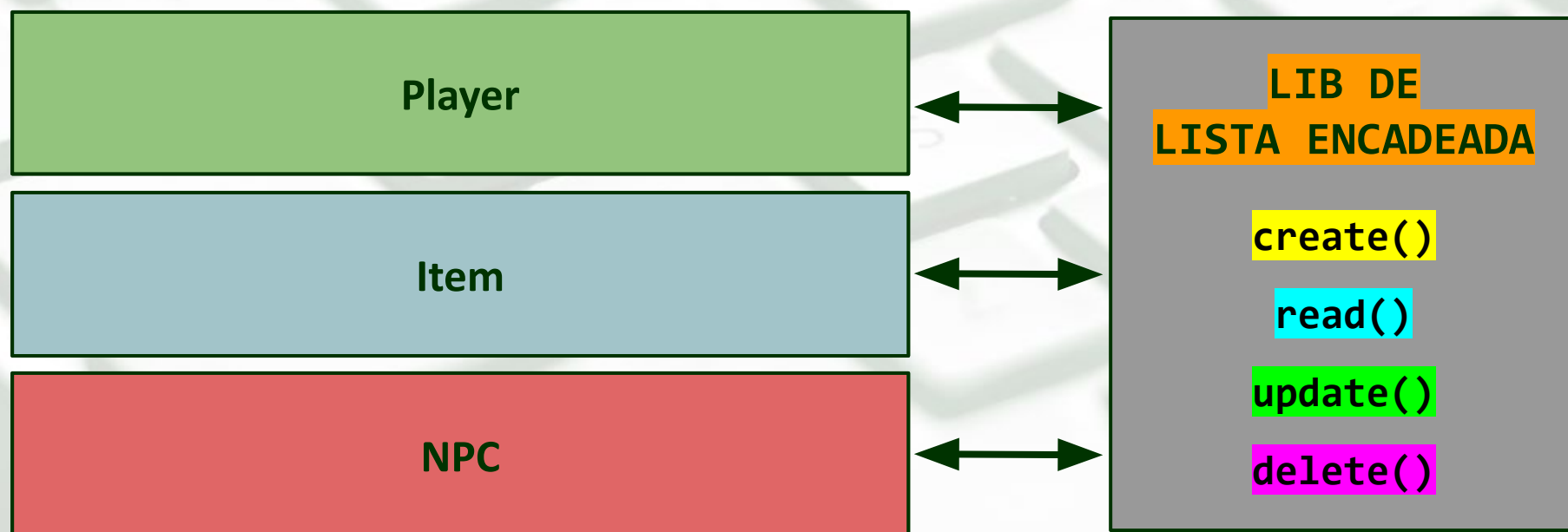
- “No mundo ideal” o mais interessante é que as operações sobre a Estrutura de Dados **Lista Encadeada** estejam **INDEPENDENTES** às entidades que a utiliza (modelos).





Listas Genéricas

- Para isso, podemos implementar uma **Biblioteca de Lista Genérica**, onde a **Estrutura básica e suas operações (CRUD)** podem ser compartilhadas por qualquer desenvolvedor e para qualquer entidade abstrata.





Conceito de Abstração

- **Abstração** é o processo de **ocultar detalhes complexos** de um sistema, **expondo apenas as funcionalidades essenciais**.
- **Abstração** permite gerenciar a complexidade, criando sistemas mais modulares, com melhor produtividade e mais fáceis de compreender.

Para dirigir um carro, você não precisa saber exatamente como o motor e suas engrenagens funcionam internamente.

*O volante, pedais e câmbio são **abstrações** que permitem a interação e uso do sistema (carro).*



Conceito de Abstração

- **Abstração** é o processo de **ocultar detalhes complexos** de um sistema, **expondo apenas as funcionalidades essenciais**.
- **Abstração** permite gerenciar a complexidade, criando sistemas mais modulares, com melhor produtividade e mais fáceis de compreender.

*De modo análogo, um **desenvolvedor** não precisa conhecer/refazer todos os detalhes internos que mantêm uma lista encadeada coesa. Ele deseja apenas efetuar operações sobre a lista, de modo íntegro e simplificado.*



Conceito de Abstração

- Entretanto, camadas de abstração sempre introduzem:
 - **Maior *Overhead*:** e consumo de recursos, devido a necessidade de tradução/conversão das chamadas (de algo simples => algo complexo).
 - **Perda do “controle fino” das operações:** Ao usar abstrações, o desenvolvedor perde o controle fino sobre as otimizações que poderiam ser feitas em baixo nível. Quanto maior o nível de abstração (alto nível), mais complexo/impraticável é descer ao baixo nível.

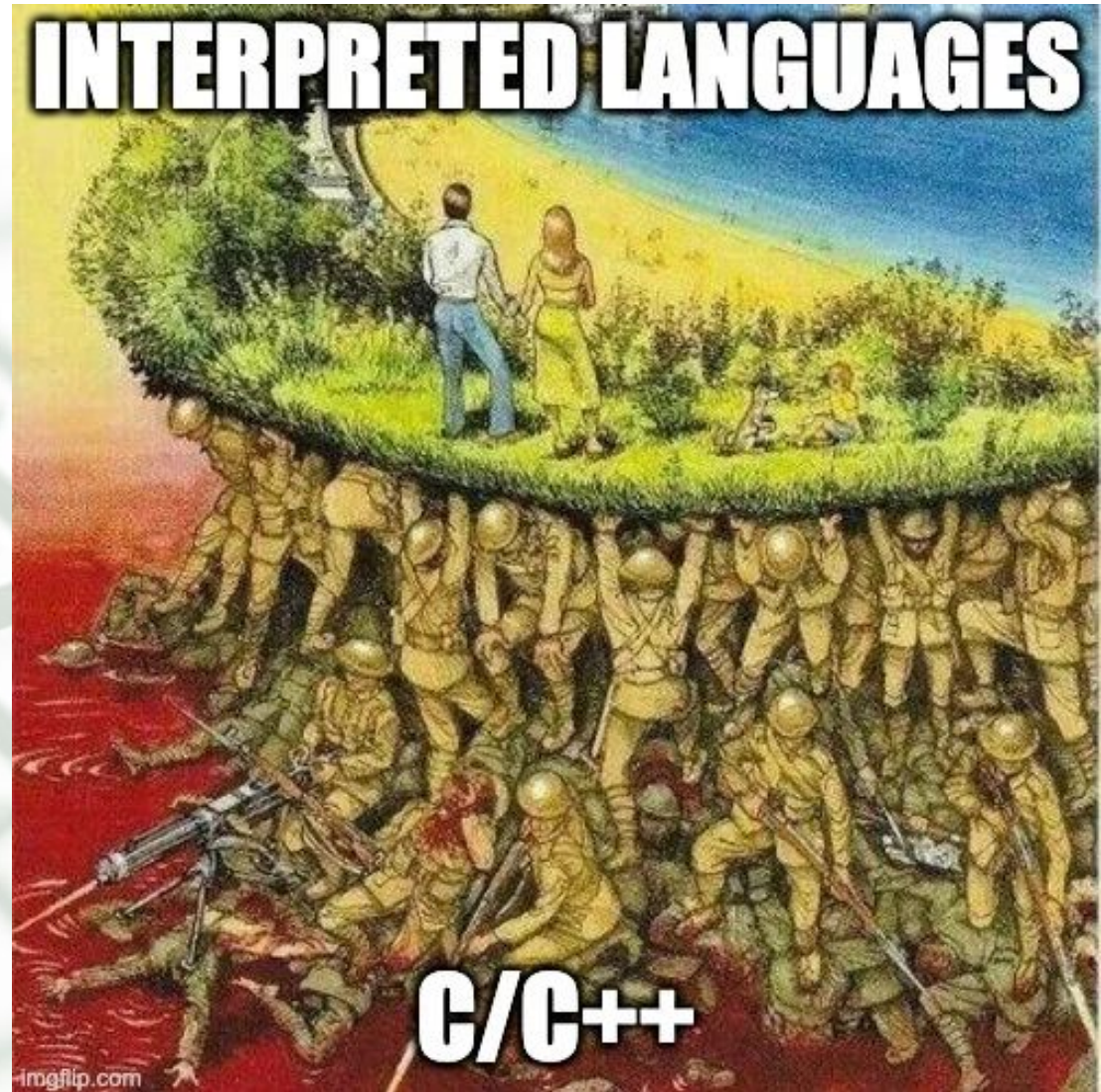


Conceito de Abstração

Produtividade

vs.

Overhead





1º Desafio

- Como evitar o alto acoplamento sendo que, na própria entidade, definimos o tipo de lista a ser utilizada...

```
typedef struct Player{  
    int id;  
    char* nome;  
    int hp;  
    struct Player* next;  
}Player;
```



1º Desafio

- Como evitar o alto acoplamento sendo que, na própria entidade, definimos o tipo de lista a ser utilizada...

```
typedef struct Player{  
    int id;  
    char* nome;  
    int hp;  
    struct Player* next;  
}Player;
```

JÁ COMEÇOU ERRADO!

Se a ideia é separar a entidade (Player) da estrutura de dados (Lista Encadeada) não podemos deixar esses dois conceitos tão acoplados!



Objetos Genéricos

■ Como definir então um objeto genérico???

```
typedef struct Node{  
    void* item;  
    struct Node* left;  
    struct Node* right;  
}Node;
```



Objetos Genéricos

■ Como definir então um objeto genérico???

```
typedef struct Node{  
    void* item;  
    struct Node* left;  
    struct Node* right;  
}Node;
```





Objetos Genéricos

■ Como definir então um objeto genérico???

```
typedef struct Node{  
    void* item;  
    struct Node* left;  
    struct Node* right;  
}Node;
```

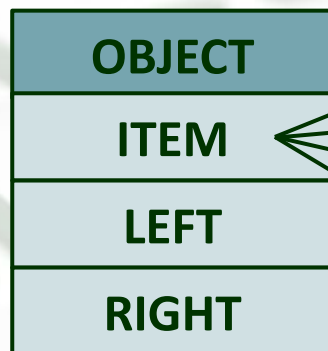
```
typedef Node* Object;
```

*Como criaremos **NODE's** de forma **dinâmica**, chamaremos esse tipo de **Object***



Objetos Genéricos

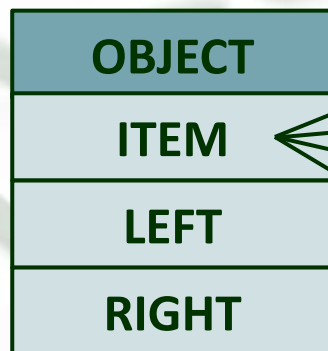
OBJECT é portanto, um PONTEIRO para uma struct NODE, que por sua vez, aponta para uma estrutura genérica útil!





Objetos Genéricos

OBJECT é portanto, um PONTEIRO para uma struct NODE, que por sua vez, aponta para uma estrutura genérica útil!



PS... Já pensou como é possível existir linguagens de tipagem fraca???



Objetos Genéricos

OBJECT é portanto, um PONTEIRO para uma struct NODE, que por sua vez, aponta

Clique e leia depois...

How Python objects are implemented in C?



Xinzhe Li, PhD in Language Intelligence · Follow
2 min read · Jul 10, 2022

RIGHT

PS... Já pensou como é possível existir linguagens de tipagem fraca???





Construtor de Objetos

■ Construindo **Objetos Genéricos** através de uma Interface Padrão.

```
#define new(TYPE,...) new_##TYPE(__VA_ARGS__)  
  
Object new_Node(){  
    //...  
}  
  
Object new_Player(char* nome, int hp){  
    Object obj = new(Node);  
    //...  
}  
  
Object new_Item(char* nome){  
    Object obj = new(Node);  
    //...  
}
```



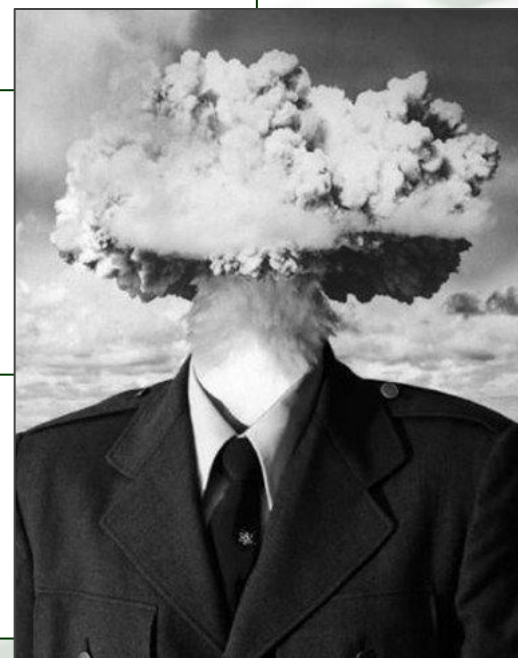
Construtor de Objetos

■ Construindo **Objetos Genéricos** através de uma **Interface Padrão.**

```
#define new(TYPE,...) new_##TYPE(__VA_ARGS__)  
  
Object new_Node(){  
    //...
```

```
int main(){  
    Object play = new(Player,"Lara Croft",100);  
    Object gun  = new(Item,"Pistol");  
}
```

```
Object new_Item(char* nome){  
    Object obj = new(Node);  
    //...  
}
```

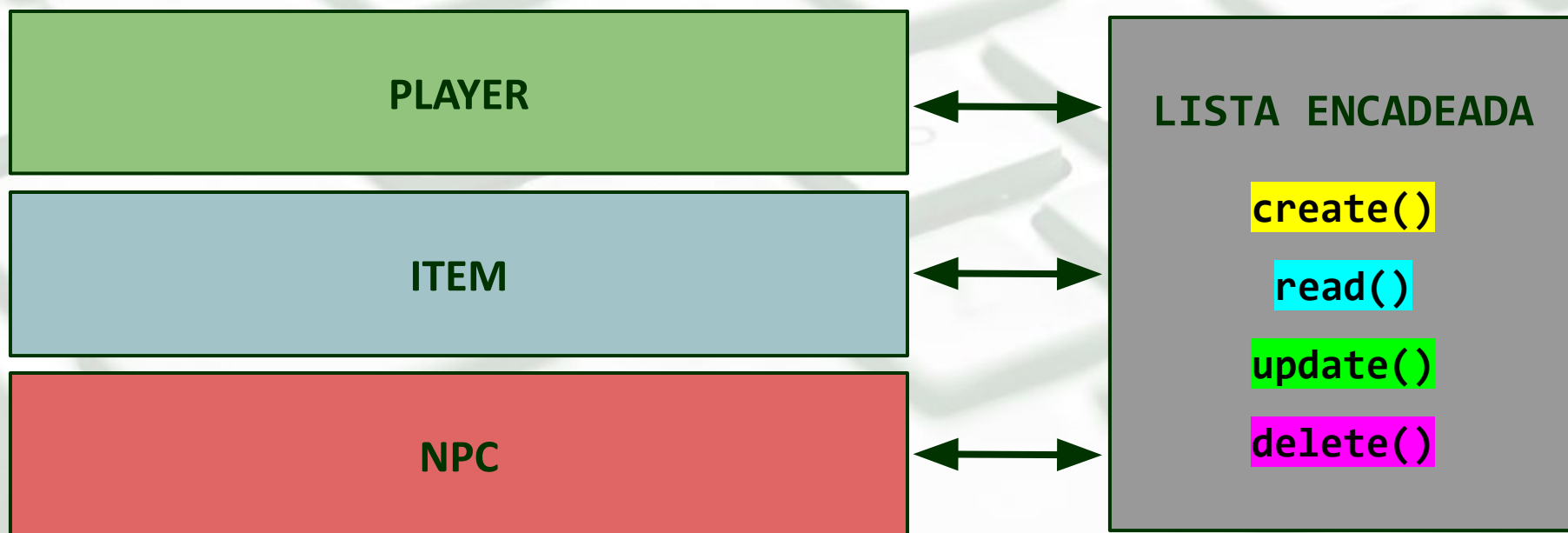




2º Desafio

- Mas um único objeto por si só não é capaz de definir toda uma lista...

DESACOPLAMENTO





Listas Genéricas

- Vamos definir então uma...

LISTA de OBJETOS GENÉRICOS

Protótipo Inicial de uma Lista Genérica

```
typedef struct Node{  
    void* item;  
    struct Node* left;  
    struct Node* right;  
}Node;
```

```
typedef Node* Object;
```

```
typedef struct{  
    Object head; //Início da lista  
    Object tail; //Final da lista  
    int size; //tamanho da lista  
    //outros dados da lista que desejar  
    //armazenar para acesso direto.  
}_List;  
typedef _List* List;
```



Listas Genéricas

■ Repositório oficial do Python (List Object)

[cpython](#) / [Include](#) / [listobject.h](#)

[vstinner](#) [gh-114329](#): Fix PyList_GetItemRef() limited C API definition (#117520)

Code

Blame

55 lines (44 loc) · 1.87 KB



Raw



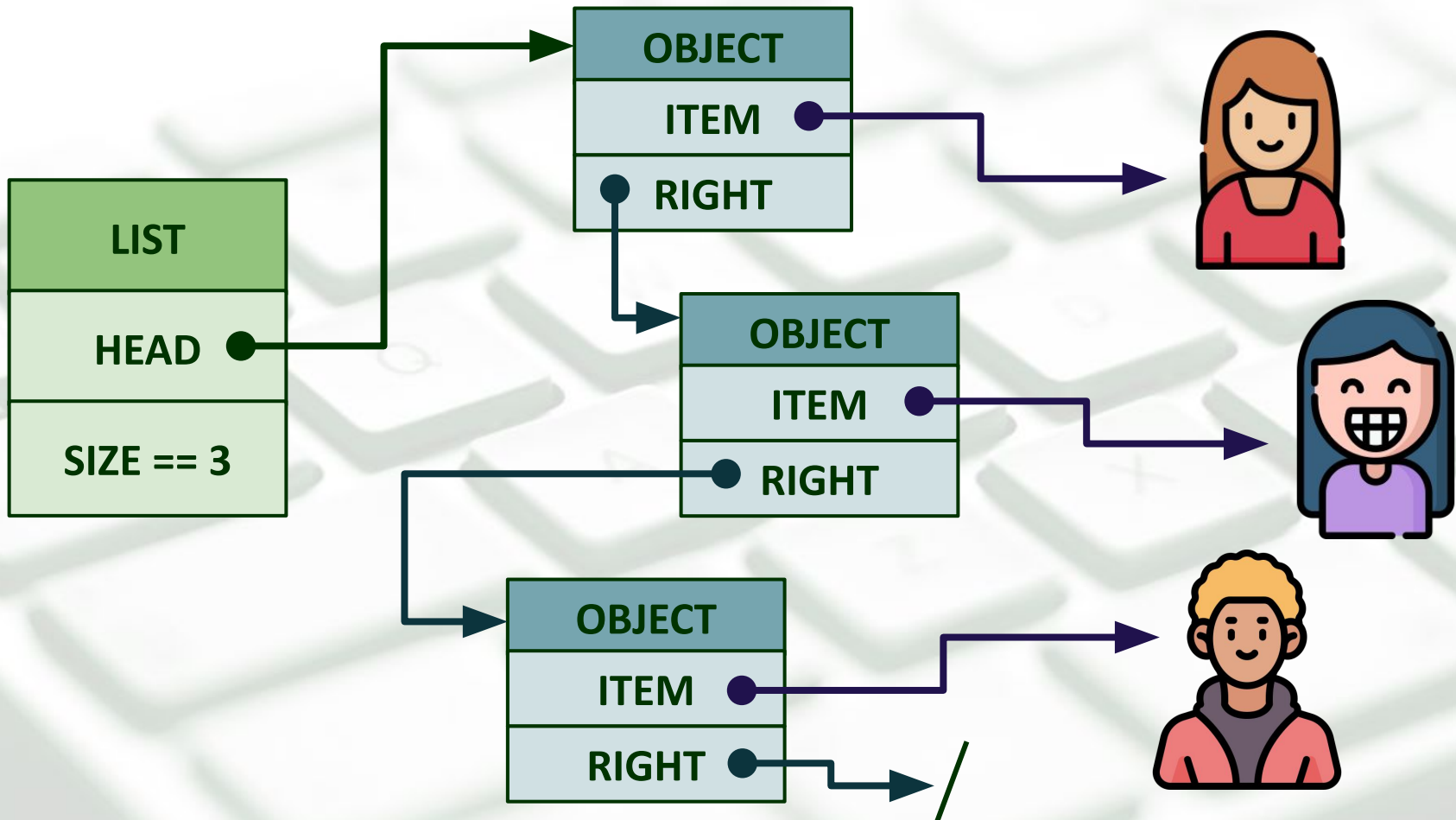
```
1  /* List object interface
2
3  Another generally useful object type is a list of object pointers.
4  This is a mutable type: the list items can be changed, and items can be
5  added or removed. Out-of-range indices or non-list objects are ignored.
6
7  WARNING: PyList_SetItem does not increment the new item's reference count,
8  but does decrement the reference count of the item it replaces, if not nil.
9  It does *decrement* the reference count if it is *not* inserted in the list.
10 Similarly, PyList_GetItem does not increment the returned item's reference
11 count.
12 */
```





Listas Genéricas v1.0

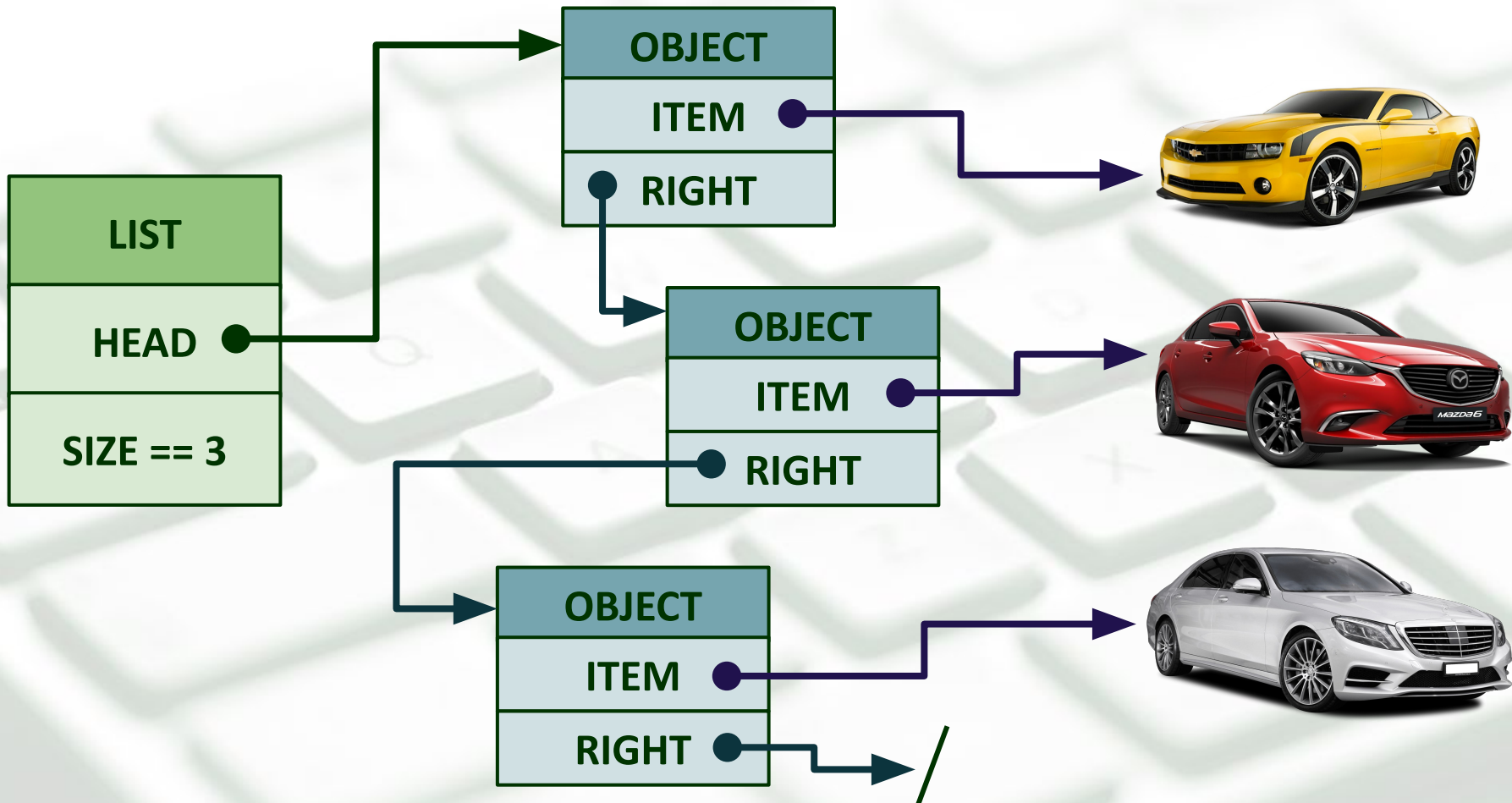
■ Implementação de Lista Genérica HOMOGÊNEA...





Listas Genéricas v1.0

■ Implementação de Lista Genérica HOMOGÊNEA...





Implementação Real

■ Exemplo de implementação em Linguagens de alto nível

```
using namespace std;

int main() {
    std::list<Carro> cars;

    cars.push_front(Carro("Territory", "Ford"));
    cars.push_front(Carro("T-Cross", "VW"));
    cars.push_front(Carro("Toro", "Fiat"));
    cars.push_front(Carro("Hilux", "Toyota"));

    for (const auto & c : cars)
        cout << c << endl;

    return 0;
}
```



Implementação Real

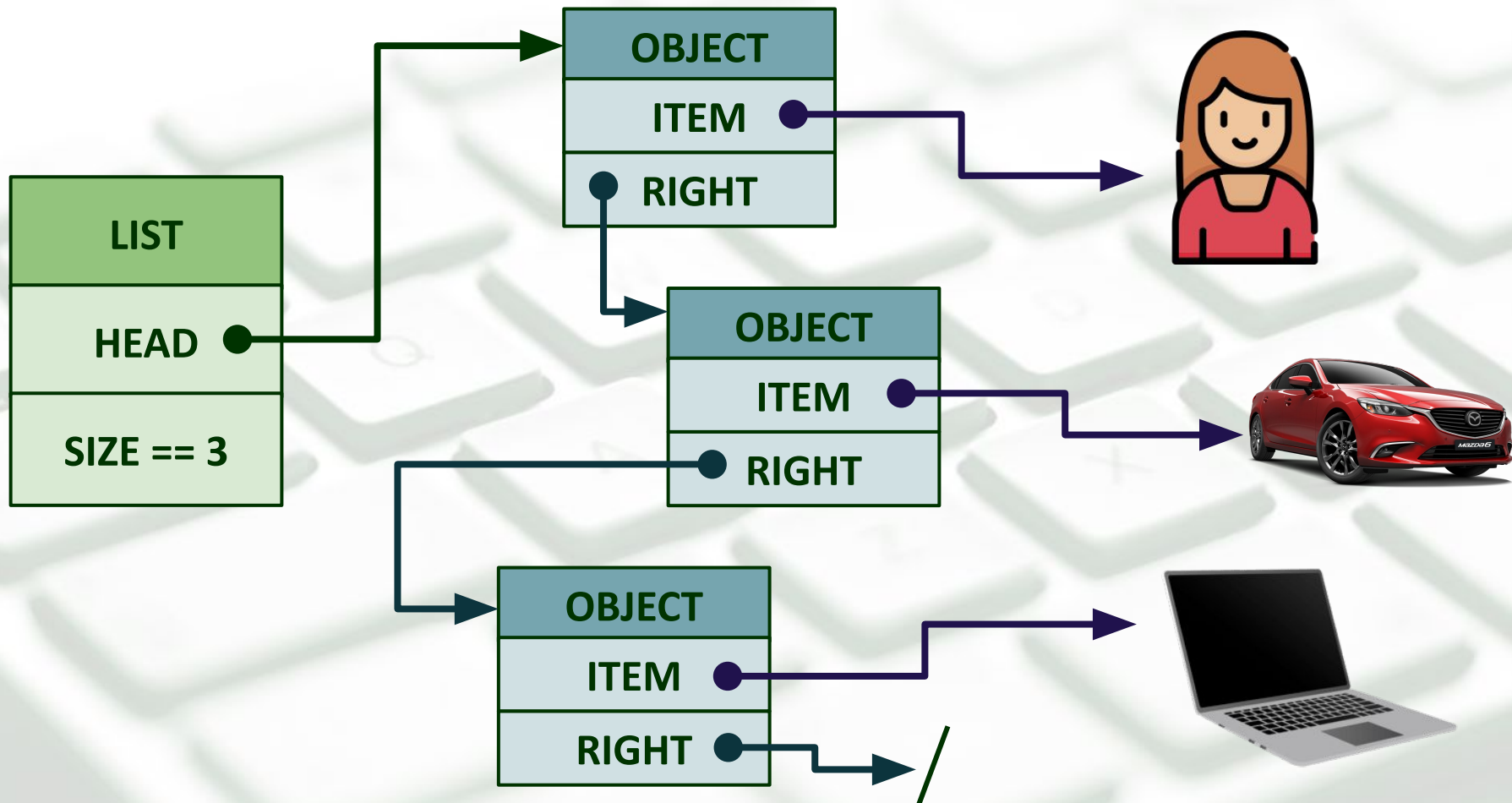
■ Exemplo de implementação em Linguagens de alto nível

```
import java.util.LinkedList;  
  
public class Main{  
    public static void main(String[] args) {  
        LinkedList<Carro> cars = new LinkedList<Carro>();  
        cars.add(new Carro("Hilux", "Toyota"));  
        cars.add(new Carro("Toro", "Fiat"));  
        cars.add(new Carro("T-Cross", "VW"));  
        cars.add(new Carro("Territory", "Ford"));  
        System.out.println(cars);  
    }  
}
```




Listas Genéricas v2.0

■ Implementação de Lista Genérica HETEROGÊNEA...





Implementação Real

■ Exemplo de implementação em Linguagens de alto nível

```
from collections import deque

def main():
    lst = deque()    #DEQUE => Double-Ended Queue
    lst.append(Carro("Hilux", "Toyota"))
    lst.append(Pessoa("João", 30))
    lst.append(Carro("T-Cross", "VW"))
    lst.append(Pessoa("Maria", "25"))

    print(list(lst))
```



■ Lista de Objetos Genéricos v1.0

- Lista duplamente encadeada, podendo ser manipulada como **Pilha** ou **Fila**.

object.h	list.h
Definição da Struct Node	Definição do Tipo List
Definição do Tipo Object	void list_push() //PILHA
Object new_Object()	void list_enqueue() //FILA
Object destroy()	Object list_pop() //RETIRA
entidade.h	void list_clear()
Definição de uma Entidade e suas funções para validação da lista.	void list_print()



Bora CODAR!!!

■ Requisitos Técnicos...

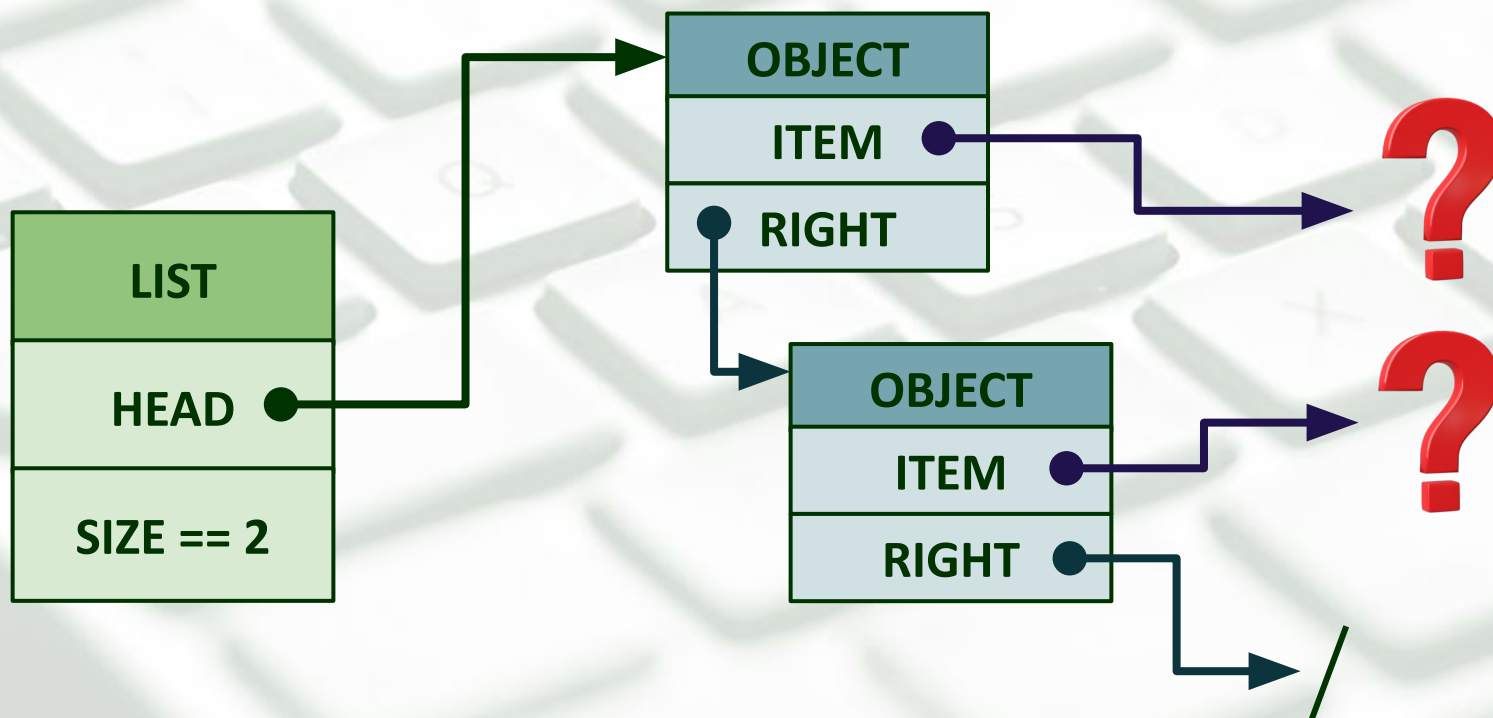
- Uso de ponteiros deverá ser transparente (SEM necessidade de *)
- Interface padronizada para construção de objetos dinâmicos.

```
int main(){
    List lst = new(List);
    Object d1 = new(Disciplina,"ED2",3,80);
    list_push(lst,d1);
    d1 = new(Disciplina,"R2",5,80);
    list_enqueue(lst,d1);
    Object ob = list_pop(lst);
    printDisciplina(ob);
    destroy(ob);
    list_print(lst);
    list_clear(lst);
    list_push(lst,new(Disciplina,"SD",6,80));
    list_print(lst);
}
```




3º Desafio

- Como os itens de uma Lista Genérica são “**indefinidos**”, como executar funções específicas (*p.ex. imprimir os dados do Nó*)?



3º Desafio

- Como o problema é "indefinido" (p.ex. i

Pesquise qual é a melhor forma de resolver esse problema.

íficas

